

```
1  class LibraryCatalog{
2      //...
3      public void updateAuthorInfo(Book b, String info){
4          try {
5              b.author.setInfo(info);
6          } catch (ReadOnlyBookException robe){
7              System.out.println(">>>Error: author info cannot be edited!");
8          }
9      }
10 }
```

Which principles is most clearly violated?

- ☐ Principle of Separation of Concerns
- ☐ Single Responsibility Principle
- ☐ Liskov Substitution Principle
- ☐ Open Closed Principle
- ☐ Law of Demeter

Q₁

```
1  class LibraryCatalog{
2      //...
3      public void updateAuthorInfo(Book b, String info){
4          try {
5              b.author.setInfo(info);
6          } catch (ReadOnlyBookException robe){
7              System.out.println(">>>Error: author info cannot be edited!");
8          }
9      }
10 }
```

Which principles is most clearly violated?

- | | | |
|-----|-------|-----|
| Yes | Maybe | No |
| Yes | Maybe | No |
| Yes | Maybe | N/A |
| Yes | Maybe | N/A |
| Yes | Maybe | No |

- ☐ Principle of Separation of Concerns
- ☐ Single Responsibility Principle
- ☐ Liskov Substitution Principle
- ☐ Open Closed Principle
- ☐ Law of Demeter

Separate the code into distinct sections, such that each section addresses a separate concern.

A class should have one, and only one, reason to change.

Derived classes must be substitutable for their base classes.

A module should be *open* for extension but *closed* for modification.

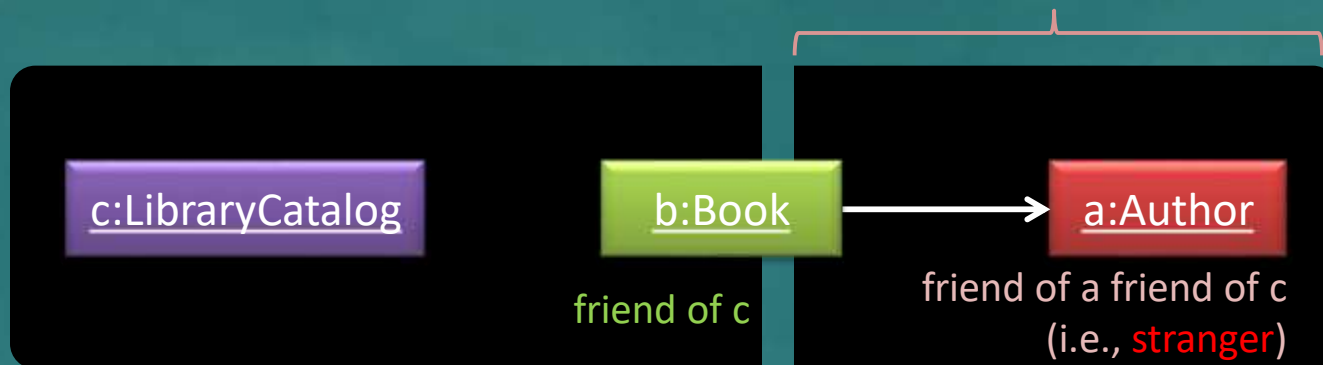
Don't talk to strangers

More on LoD violations ...

```
1  class LibraryCatalog{
2      //...
3      public void updateAuthorInfo(Book b, String info){
4          try {
5              b.author.setInfo(info); b.getAuthor().setInfo(); // same problem (but slightly better)!
6          } catch (ReadOnlyBookException robe){
7              System.out.println(">>>>Error: author info cannot be edited!");
8          }
9      }
10 }
```

☹ LibraryCatalog has a dependency on the internal object structure of Book!
What if we wanted to move the author object to a different location?

i.e., Do not navigate inside the **internal object structure**!



When paying a delivery person, do we allow that person to reach into your pocket, take your wallet out, and take money from it?

NO !!!

Alternatives:

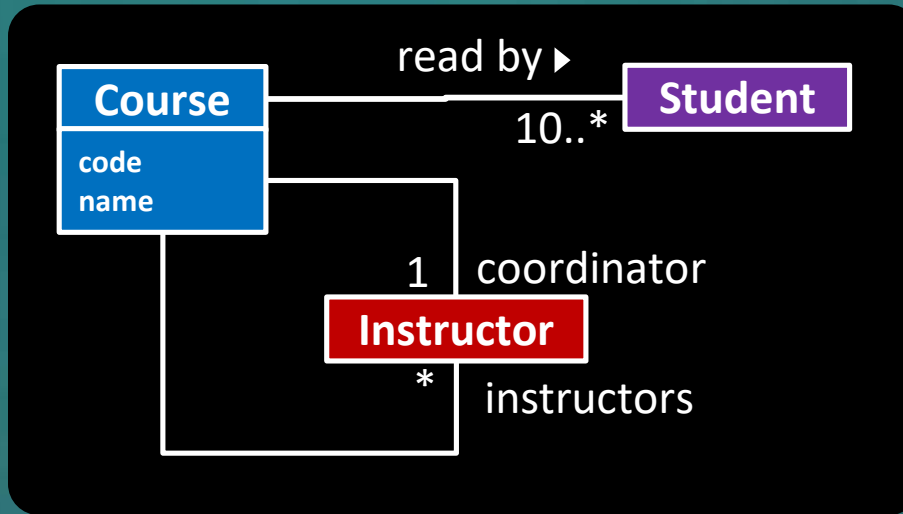
- Receive an Author object instead of a Book
- Ask the Book to change author info.

Don't talk to strangers

Perhaps OK, as the method name clearly states it. But try to minimize.

Q₂

Which statements about this CCD are correct?



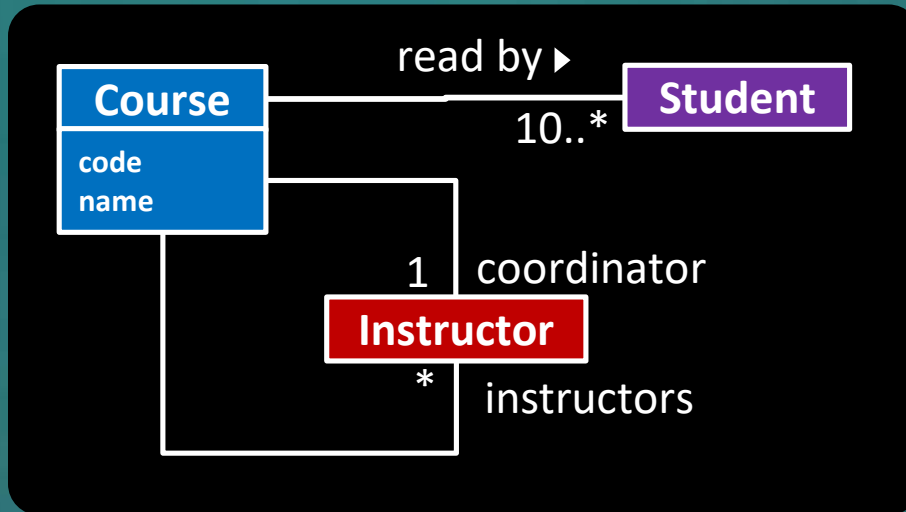
CCD is a model of the real world.

The design and the implementation of the solution can be guided by it, but it's not the same.

→ **avoid implementation-specific terminology** when discussing the problem domain.

- ☐ A. The **instructors** variable in the **Course** class can be of type **ArrayList<Instructor>**
- ☐ B. The **Course** class can have methods to add students to a specific course.
- ☐ C. There are alternative designs that can be more efficient to implement.
- ☐ D. Although not shown in the diagram, it is likely the **Student** class has attributes such as name, student number, gender etc.
- ☐ E. Instructor objects can play the *role* of coordinator.

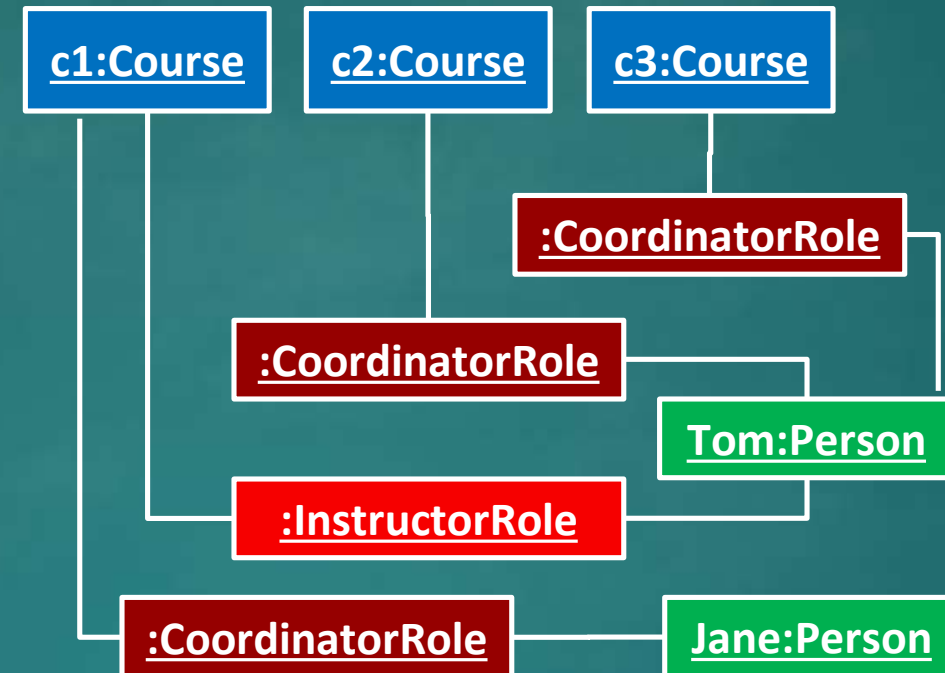
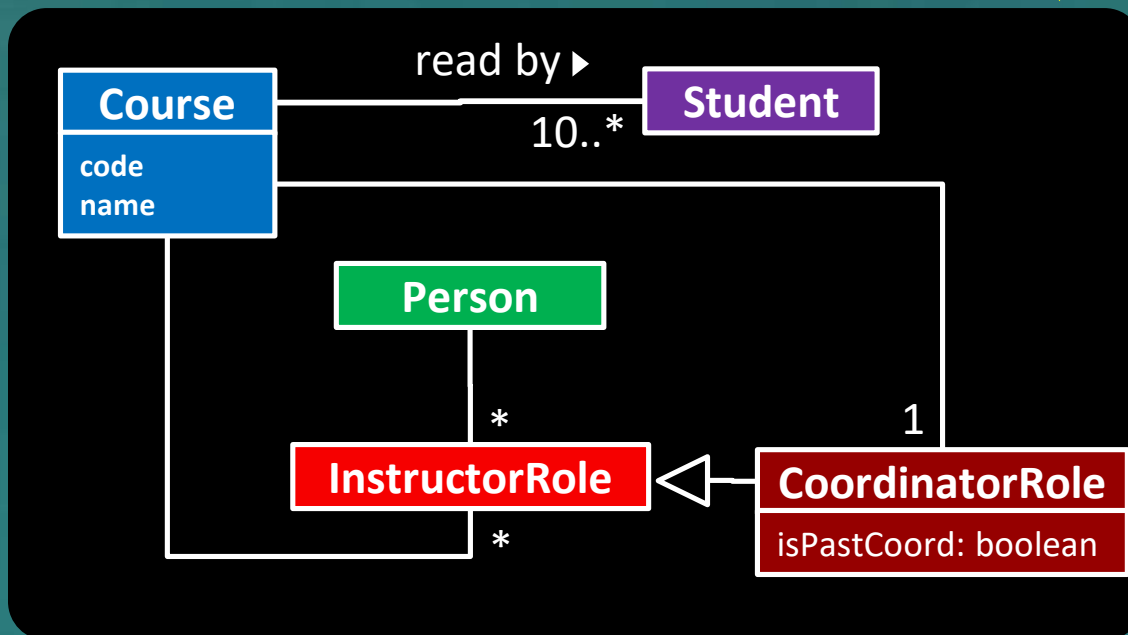
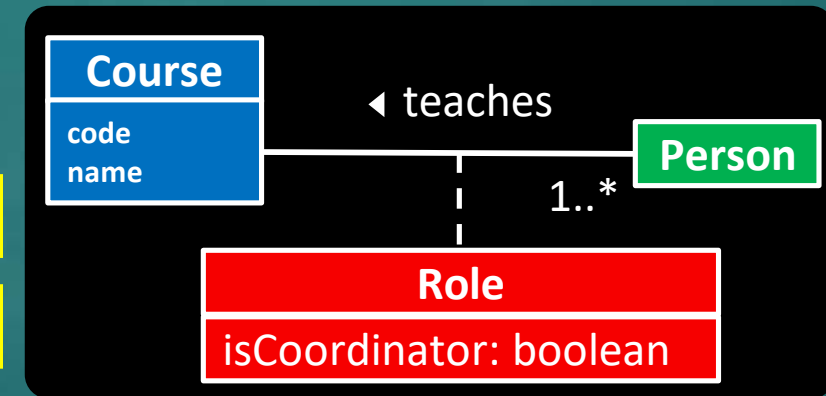
A course has a name and a code. A course is read by 10 or more students, and taught by one or more instructors one of whom is the coordinator.



Multiple acceptable ways to model a domain!

Simpler

More flexible

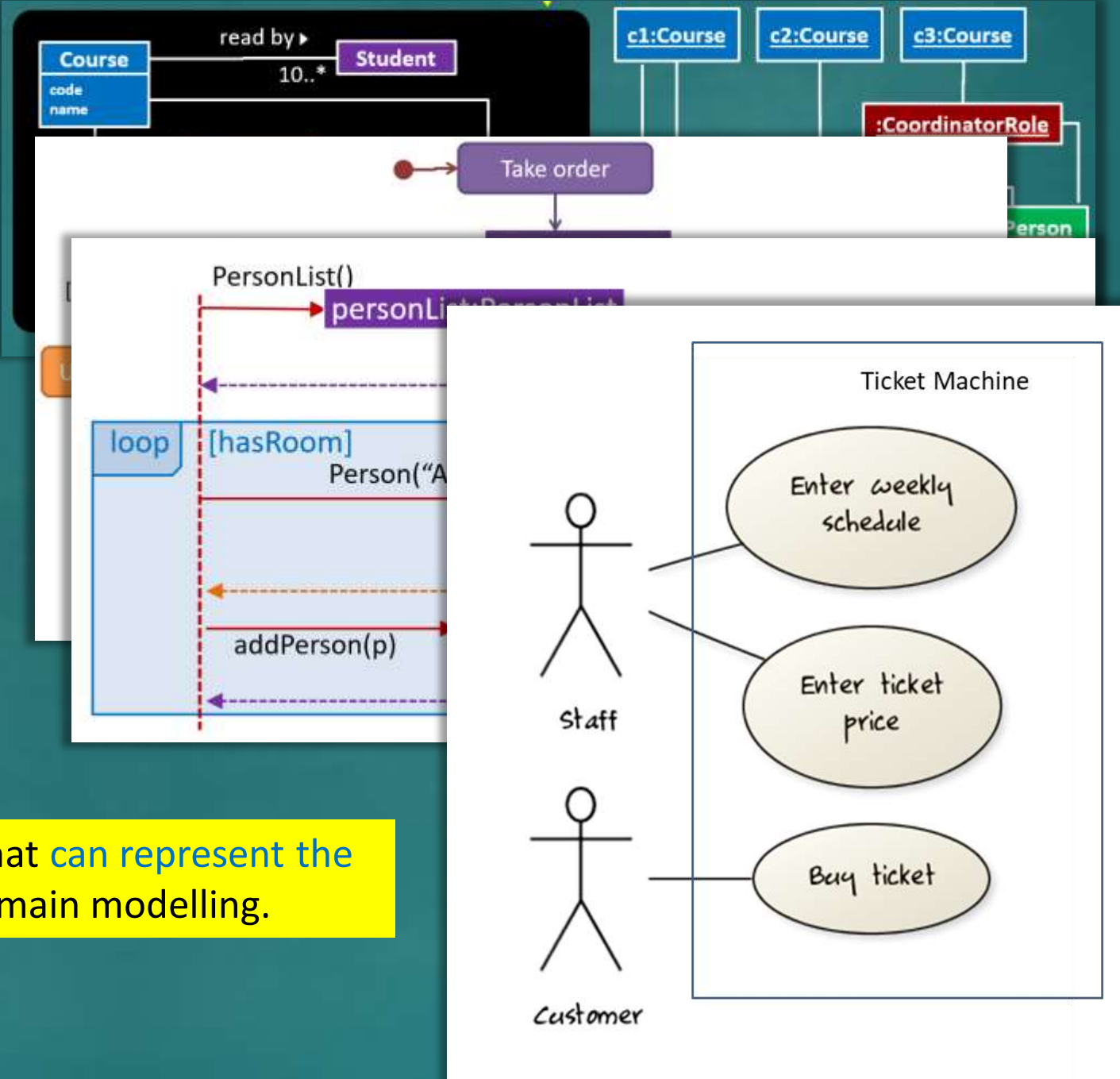


Q₃

Which can be used for *domain modelling*?

- ☐ A CCD
- ☐ An object diagram
- ☐ An activity diagram
- ☐ A sequence diagram
- ☐ A class diagram
- ☐ A use case diagram

Anything (not just UML diagrams) that **can represent the problem domain** can be used for domain modelling.



Q₄

Which SDLC process model is used by Google?

- ☐ Scrum
- ☐ XP
- ☐ Unified process
- ☐ None of the above

Some companies 'claim' to follow a specific process.

Many companies do use specific parts
e.g., *daily scrum*.

Most companies have their own process(es) that may borrow from these process models.

Q₅

The 'Quick start' section of your tP UG belongs to which category?

TUTORIALS A tutorial: • is learning-oriented • allows the newcomer to get started • is a lesson Analogy: teaching a small child how to cook	HOW-TO GUIDES A how-to guide: • is goal-oriented • shows how to solve a specific problem • is a series of steps Analogy: a recipe in a cookery book
EXPLANATION An explanation: • is understanding-oriented • explains • provides background and context Analogy: an article on culinary social history	REFERENCE A reference guide: • is information-oriented • describes the machinery • is accurate and complete Analogy: a reference encyclopedia article

Quick start

1. Ensure you have Java 11 or above installed in your C
2. Download the latest `addressbook.jar` from [here](#).
3. Copy the file to the folder you want to use as the *home*
4. Double-click the file to start the app. The GUI similar to
Note how the app contains some sample data.

 Address App

File Help

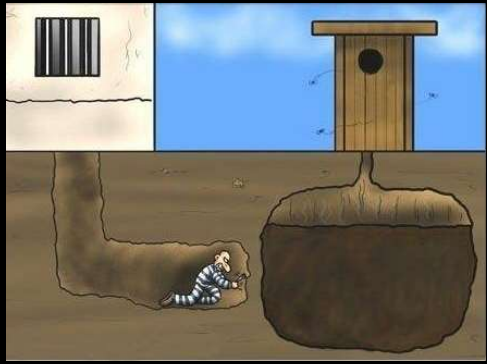
Enter command here...

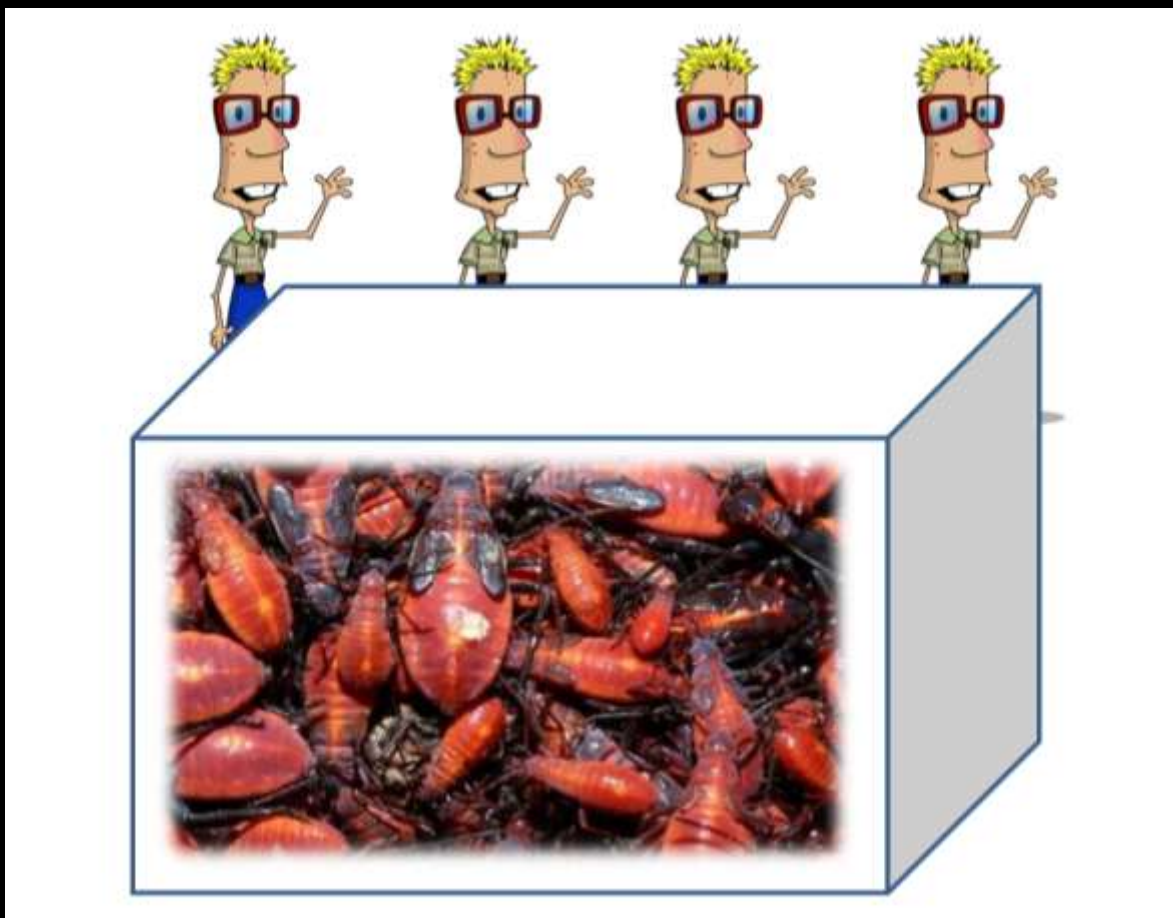
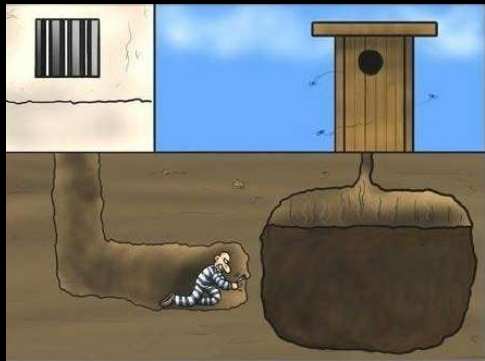
Try to keep each doc within one category,
although minor cross-overs may be unavoidable.

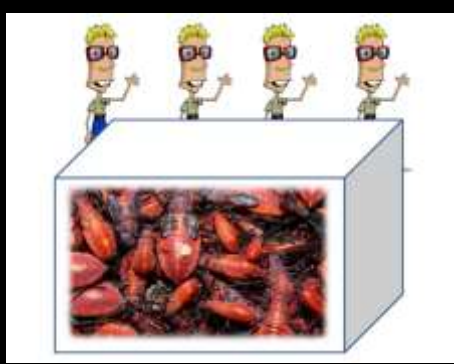
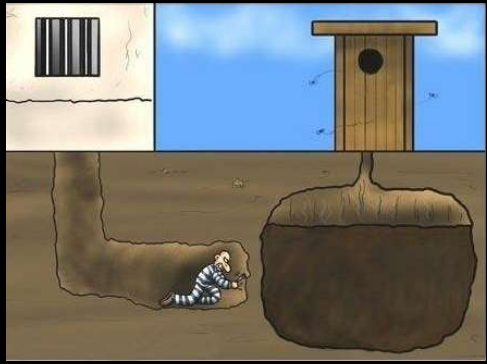
v1.3

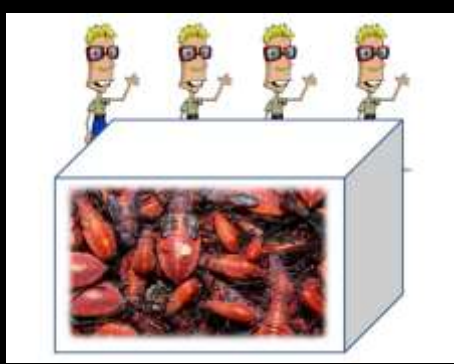
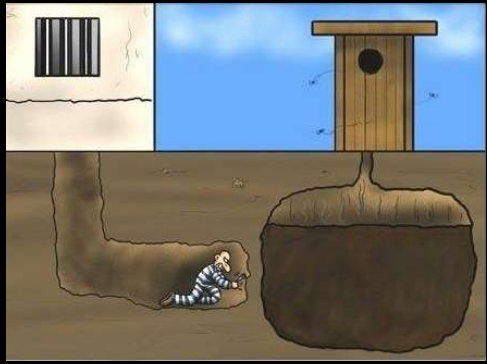
How?

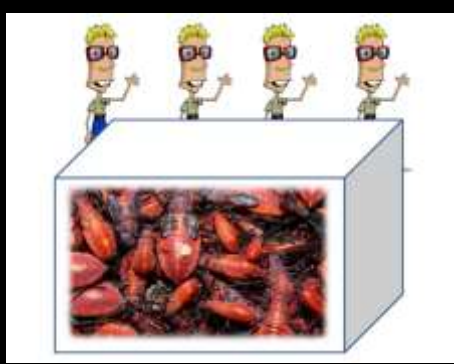
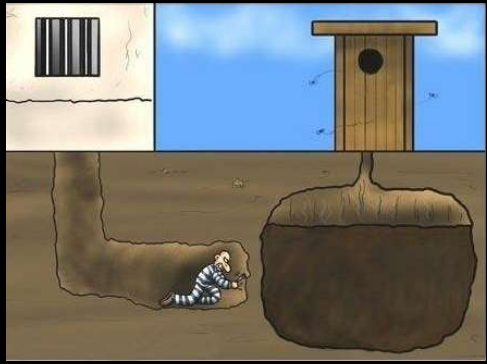


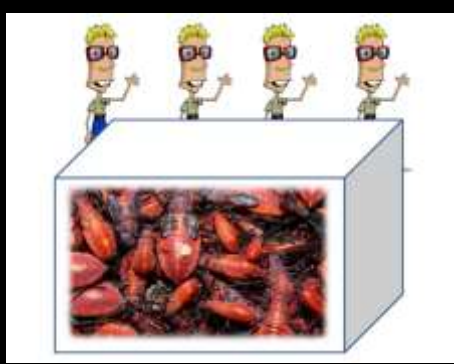
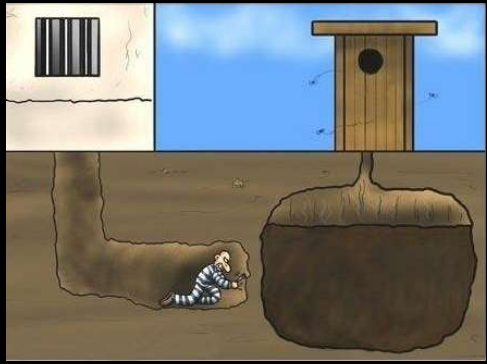


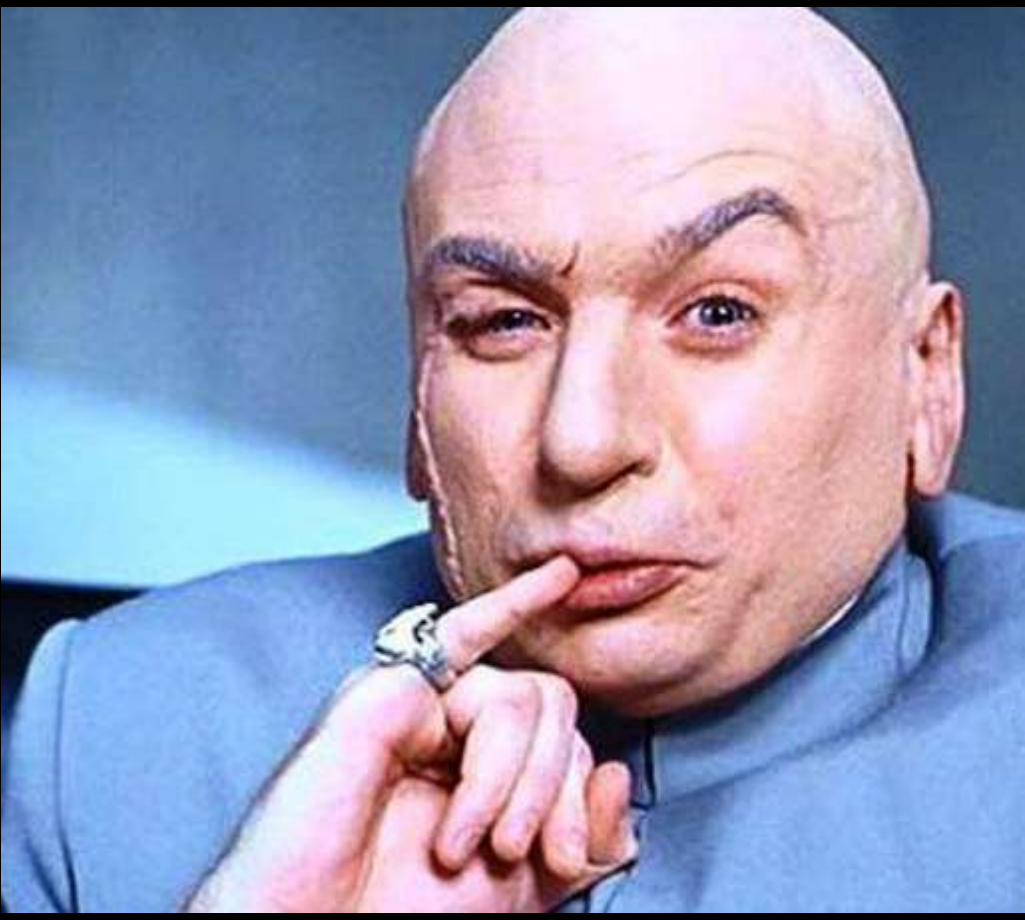
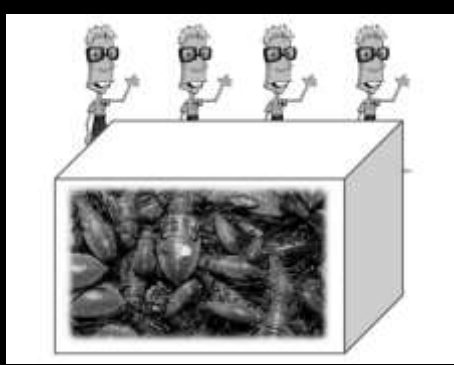
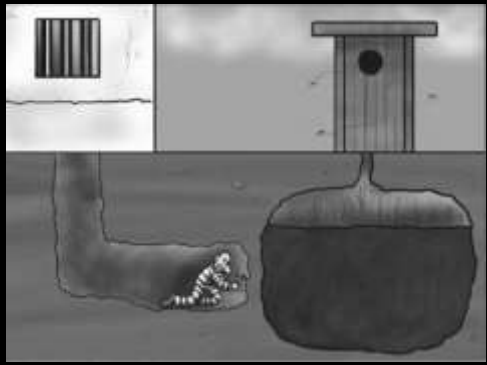




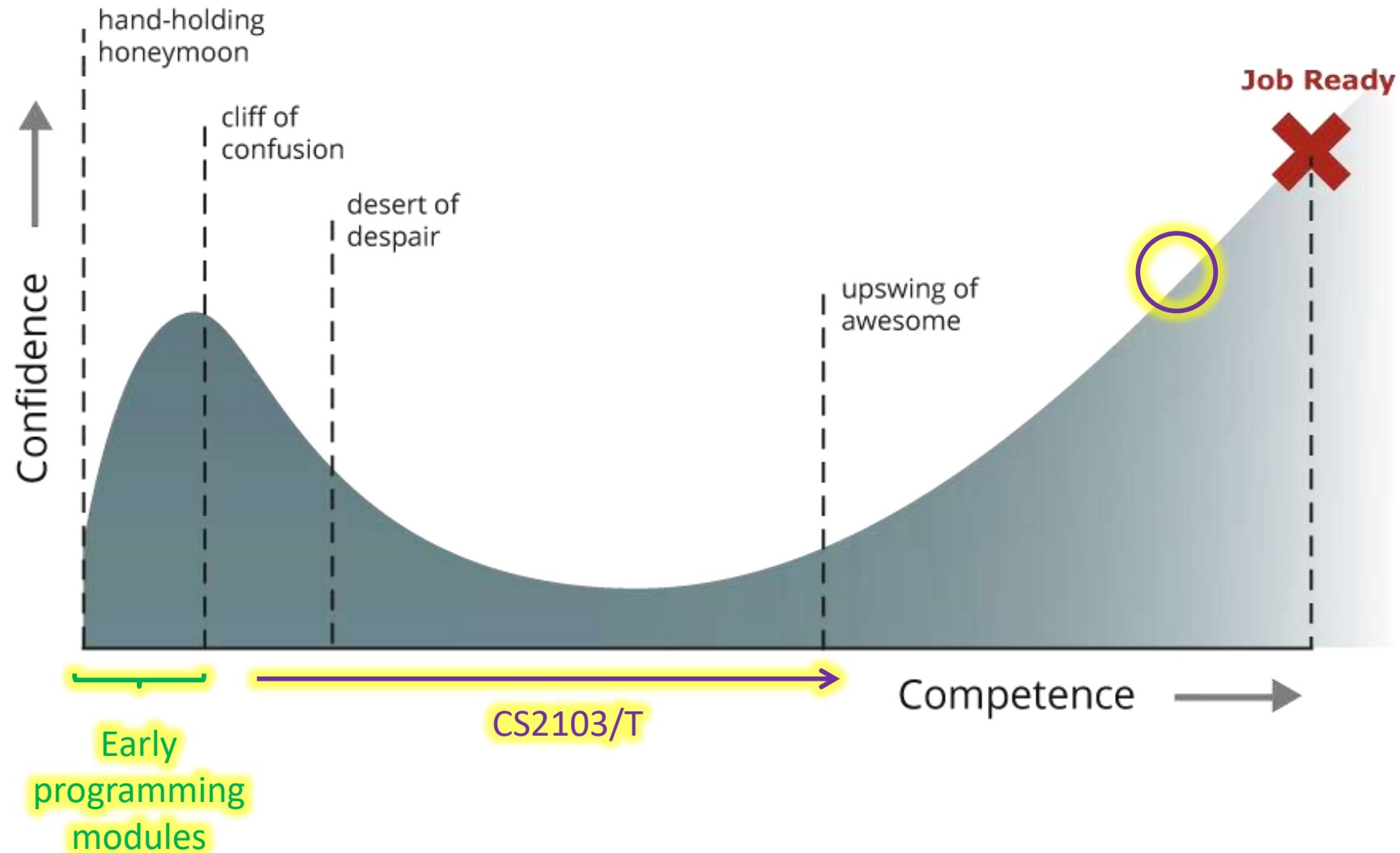








Coding Confidence vs Competence



v1.3



v1.6



v1.3



v1.6



v1.3



v1.6



v1.2



v1.6



1. Aim to create a gem
you will be proud to
wear



2. Polish, check, polish,
check, polish, check,...



3. Sharpen your tools
once in a while



Week 10

Topics:

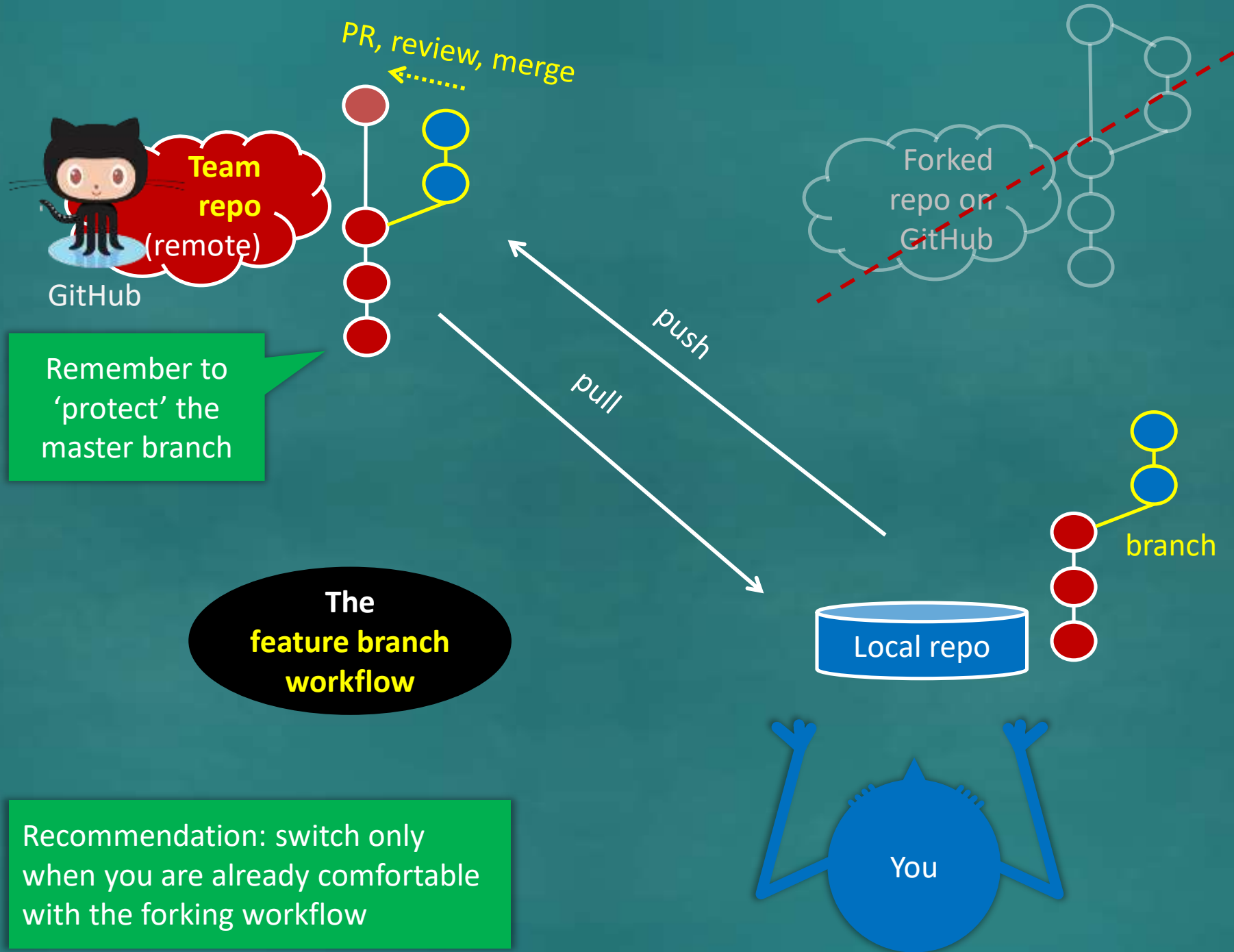
- [W10.1] **Design Patterns** ⚡⚡
- [W10.2] **Defensive Programming** ⚡⚡
- [W10.3] **Test Cases: Intro** ⚡⚡
- [W10.4] **Test Cases: Equivalence Partitioning** ⚡⚡
- [W10.5] **Test Cases: Boundary Value Analysis** ⚡⚡

Admin:

1. Submit weekly quiz P
2. Take part in the CATcher load testing
⌚ during the briefing on Mar 28th P

tP: 📁 v1.4

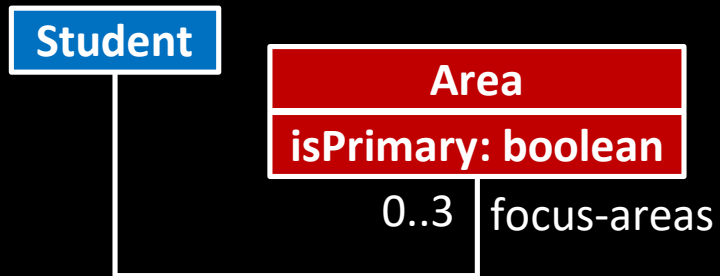
1. 👥 Do a postmortem of the previous iteration
2. 👥 Plan the alpha version (v1.4)
3. 👥 Deliver the alpha version (v1.4)
4. 👤 Smoke-test CATcher ⌚ COMPULSORY P
5. 👤 Start updating UML diagrams in the DG



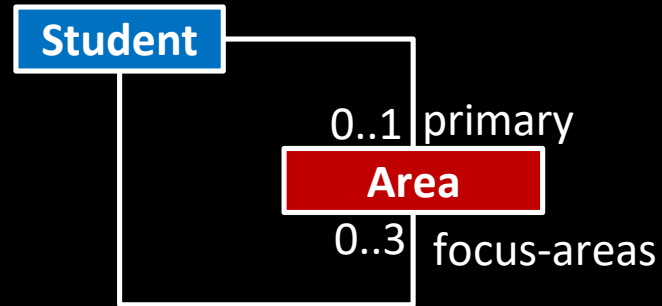
Choose the CCD that matches this description best.

Each student chooses up to 3 focus areas, one of which can be selected as the primary focus area.

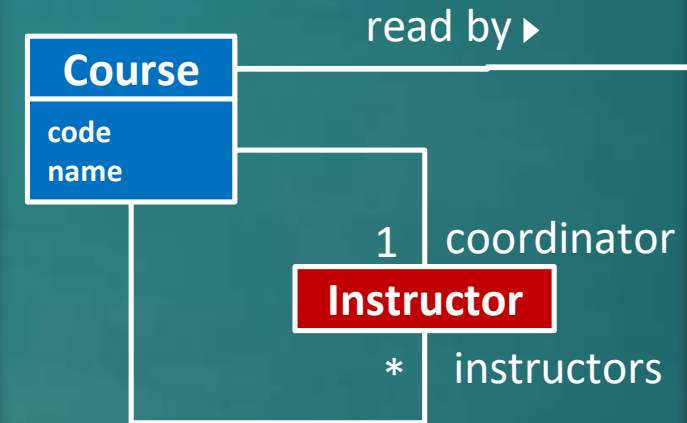
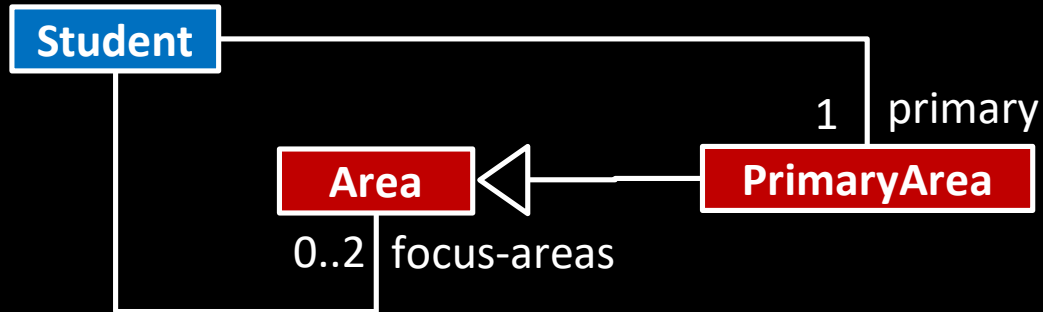
(a)



(b)



(c)



Note how the experience from the tutorial helped you 'pattern match' a solution here.

Design patterns:

Singleton, Façade, Model-View-Controller etc.

Design principles: SOLID, LoD, SoC, etc.

Fundamental design concepts:

Abstraction, Coupling, Cohesion

Week 10

Topics:

- [W10.1] **Design Patterns** ⚡⚡
- [W10.2] **Defensive Programming** ⚡⚡
- [W10.3] **Test Cases: Intro** ⚡⚡
- [W10.4] **Test Cases: Equivalence Partitioning** ⚡⚡
- [W10.5] **Test Cases: Boundary Value Analysis** ⚡⚡

Admin:

1. Submit weekly quiz **P**
2. Take part in the CATcher load testing
⌚ during the briefing on Mar 28th **P**

tP: **v1.4**

1. 👤 Do a postmortem of the previous iteration
2. 👤 Plan the alpha version (v1.4)
3. 👤 Deliver the alpha version (v1.4)
4. 👤 Smoke-test CATcher **⌚ COMPULSORY** **P**
5. 👤 Start updating UML diagrams in the DG

ARIANE 5 launch



System 1: issue navigation commands

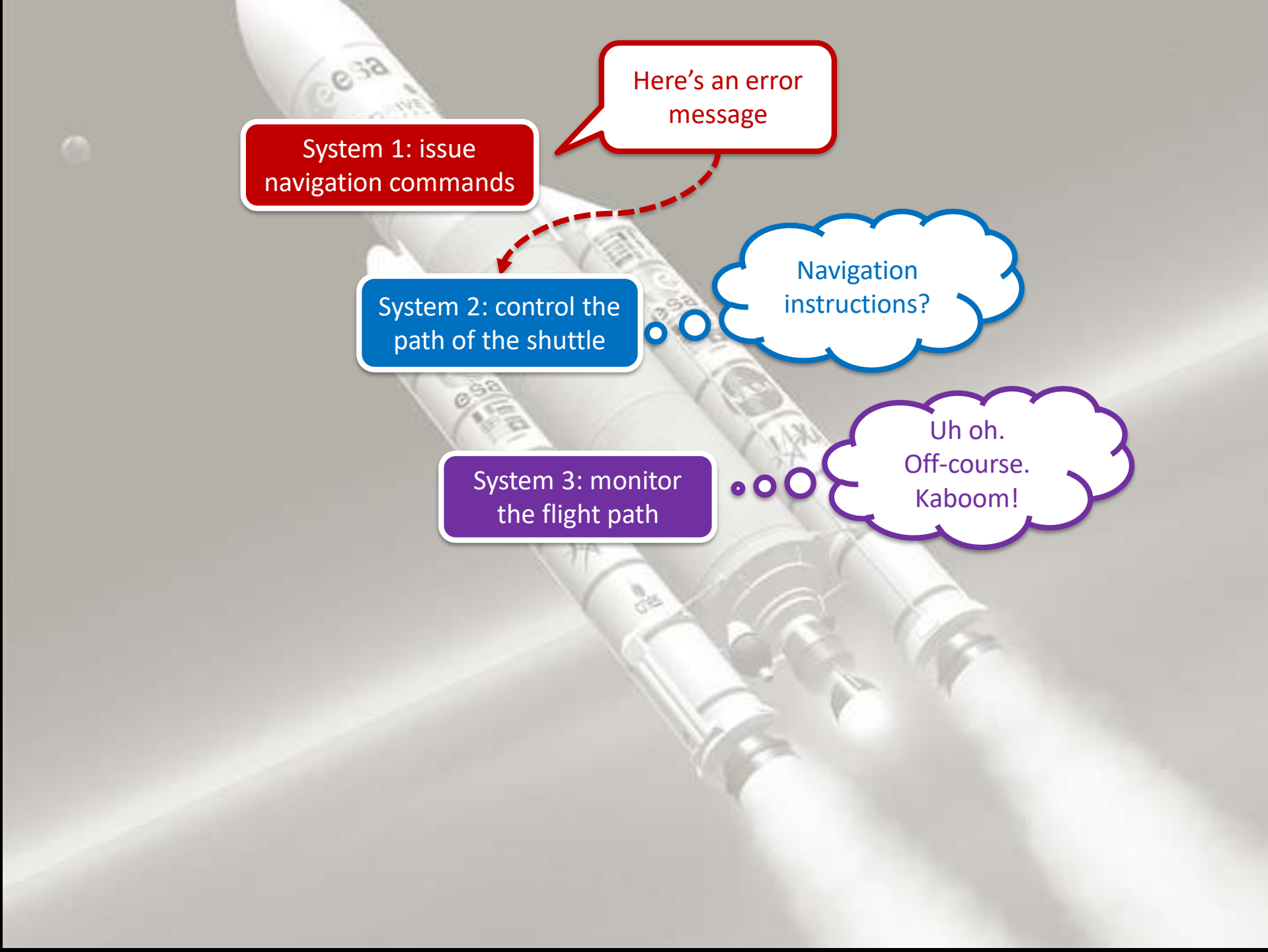
Here's an error message

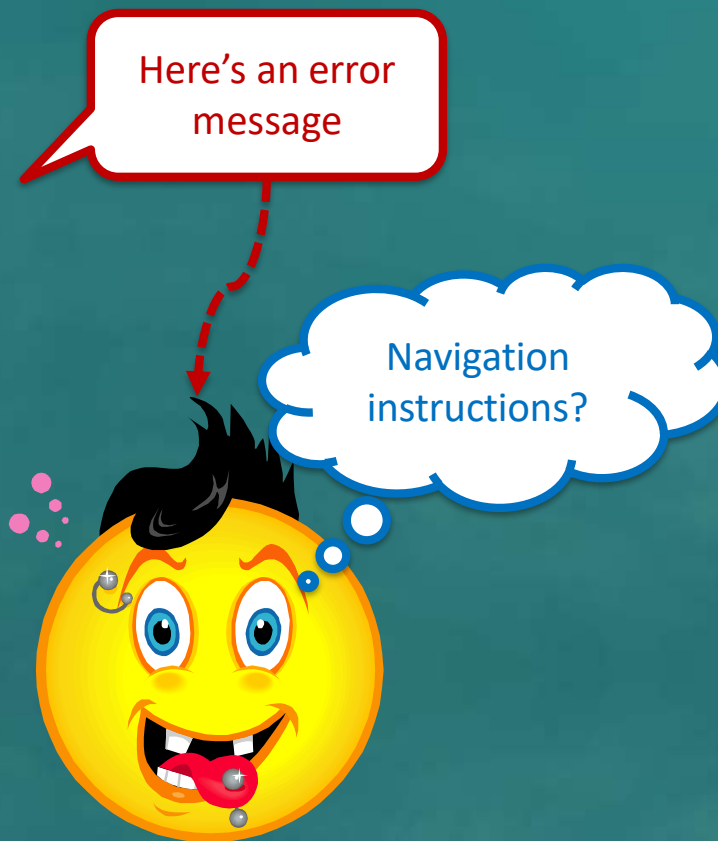
System 2: control the path of the shuttle

Navigation instructions?

System 3: monitor the flight path

Uh oh.
Off-course.
Kaboom!





Your teammate's
code



Your code

Putting up defences to protect our code.



Your teammate's
code



Your code

Putting up defences to protect our code.



Your teammate's
code

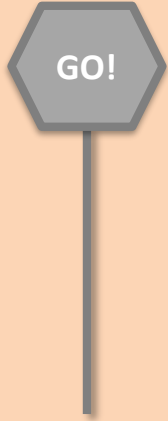


Putting up defences to protect our code.

MISUNDERSTANDINGS



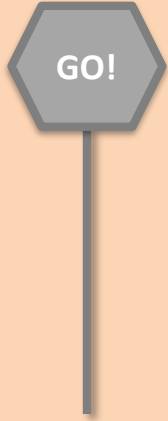
MISUNDERSTANDINGS



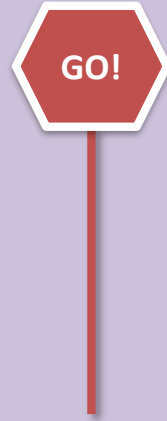
Assertions

```
x = getX();  
assert x == 0 : "x should be 0";  
...
```


MISUNDERSTANDINGS



MISHAPS



Assertions

MISUNDERSTANDINGS



Assertions

MISHAPS



Exceptions



Assertion or exception?

A developer screwed up

e.g., a method is given null as a parameter although null values are not supposed to reach that method

→ Use assertions to *abort*

A user screwed up (or a problem in the environment)

e.g., A file is not found in the expected location

→ Use exceptions to *handle*

Q

Assertion or exception?

Consider this method:

```
execute(String userCommand)
```

If the `userCommand` is in an incorrect format,
it should be handled using,

- ☐ Assertions
- ☐ Exceptions
- ☐ It depends

A developer screwed up
→ Use assertions to *abort*

A user screwed up
→ Use exceptions to *handle*

Q

Assume your app tries to contact a Google API. **If the Google servers are down**, it should be handled using,

- ☐ Assertions
- ☐ Exceptions
- ☐ It depends

MISUNDERSTANDINGS



Assertions

MISHAPS



Exceptions

MYSTERIES



MISUNDERSTANDINGS



Assertions

MISHAPS



Exceptions

MYSTERIES



Logging

MISUNDERSTANDINGS



Assertions

MISHAPS



Exceptions

MYSTERIES



Logging

MISUSE



MISUNDERSTANDINGS



Assertions

MISHAPS



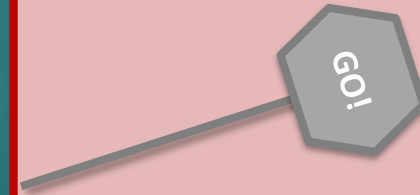
Exceptions

MYSTERIES



Logging

MISUSE



Defensive

Other three
plus more!

Use all four in your own tP code!

Week 10

Topics:

- [W10.1] Design Patterns ⚡⚡
- [W10.2] Defensive Programming ⚡⚡
- [W10.3] Test Cases: Intro ⚡⚡
- [W10.4] Test Cases: Equivalence Partitioning ⚡⚡
- [W10.5] Test Cases: Boundary Value Analysis ⚡⚡

Admin:

1. Submit weekly quiz P
2. Take part in the CATcher load testing
⌚ during the briefing on Mar 28th P

tP: 📁 v1.4

1. 👤 Do a postmortem of the previous iteration
2. 👤 Plan the alpha version (v1.4)
3. 👤 Deliver the alpha version (v1.4)
4. 👤 Smoke-test CATcher ⌚ COMPULSORY P
5. 👤 Start updating UML diagrams in the DG

Poll: of these three, which career path do you prefer?

- a. System analyst (i.e., talk to users/customers)
- b. Software engineer
- c. Software tester

[don't worry, there are other career paths too...]

Testing as a career...?

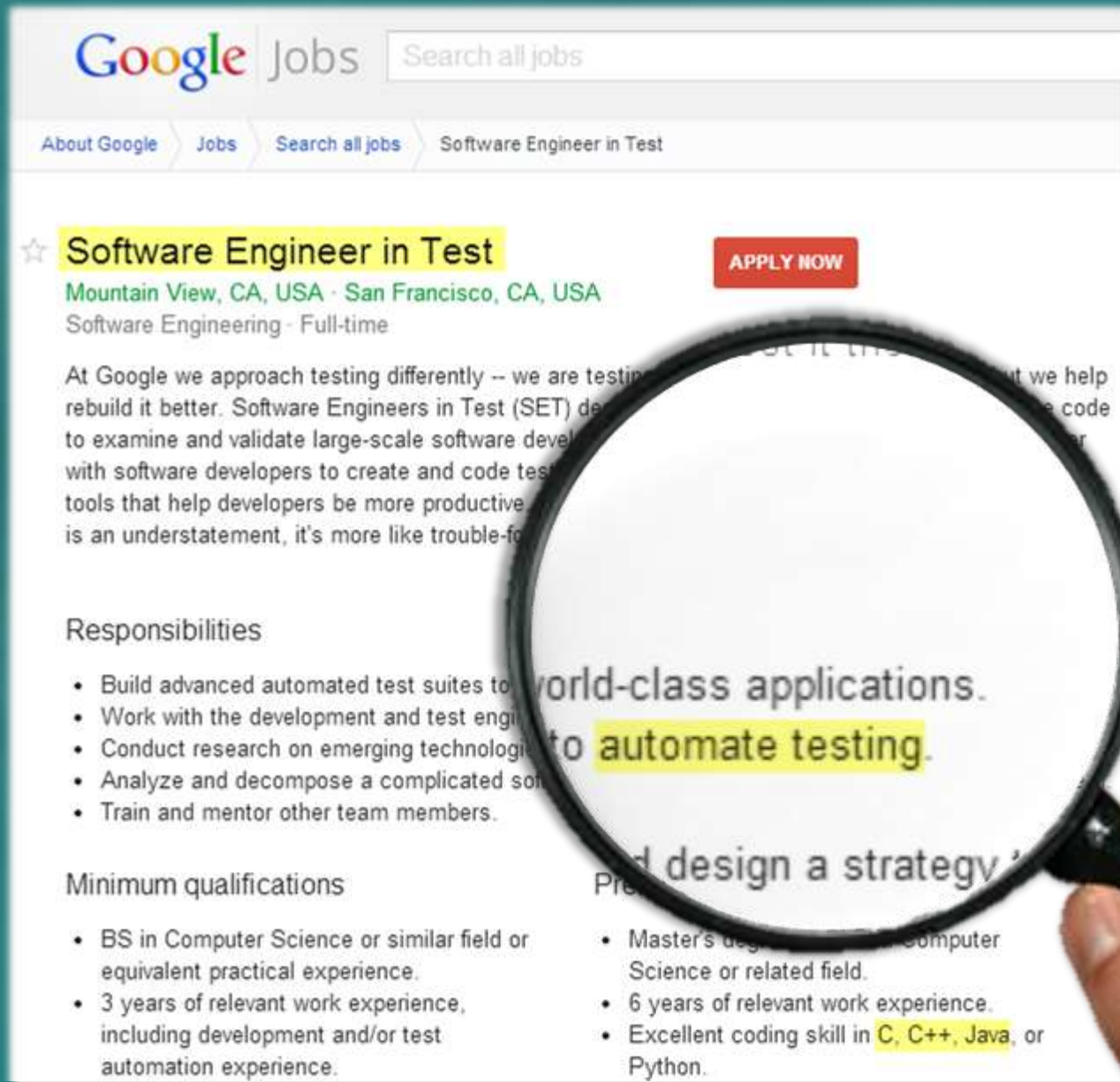
Easier to get in,
get ahead

Blame others,
not get blamed

Still can code



Testing as a career...?



The image shows a screenshot of a Google Jobs listing for a 'Software Engineer in Test' position. A magnifying glass is held over the text 'to automate testing', which is highlighted in yellow. The listing includes details about the location (Mountain View, CA, USA and San Francisco, CA, USA), the job type (Software Engineering - Full-time), and an 'APPLY NOW' button. The description mentions that at Google, testing is approached differently, focusing on rebuilding better. The responsibilities listed include building automated test suites, working with development and test engineers, conducting research on emerging technologies, analyzing and decomposing complicated software, and training/mentoring team members. The minimum qualifications section lists a BS in Computer Science or equivalent practical experience, 3 years of relevant work experience (including development and/or test automation), a Master's degree in Computer Science or related field, 6 years of relevant work experience, and excellent coding skill in C, C++, Java, or Python.

Google Jobs Search all jobs

About Google Jobs Search all jobs Software Engineer in Test

☆ **Software Engineer in Test** [APPLY NOW](#)

Mountain View, CA, USA · San Francisco, CA, USA

Software Engineering - Full-time

At Google we approach testing differently -- we are testing to rebuild it better. Software Engineers in Test (SET) design and build tools to examine and validate large-scale software development. They work with software developers to create and code test suites, and build tools that help developers be more productive. At Google, it's not an understatement, it's more like trouble-finding.

Responsibilities

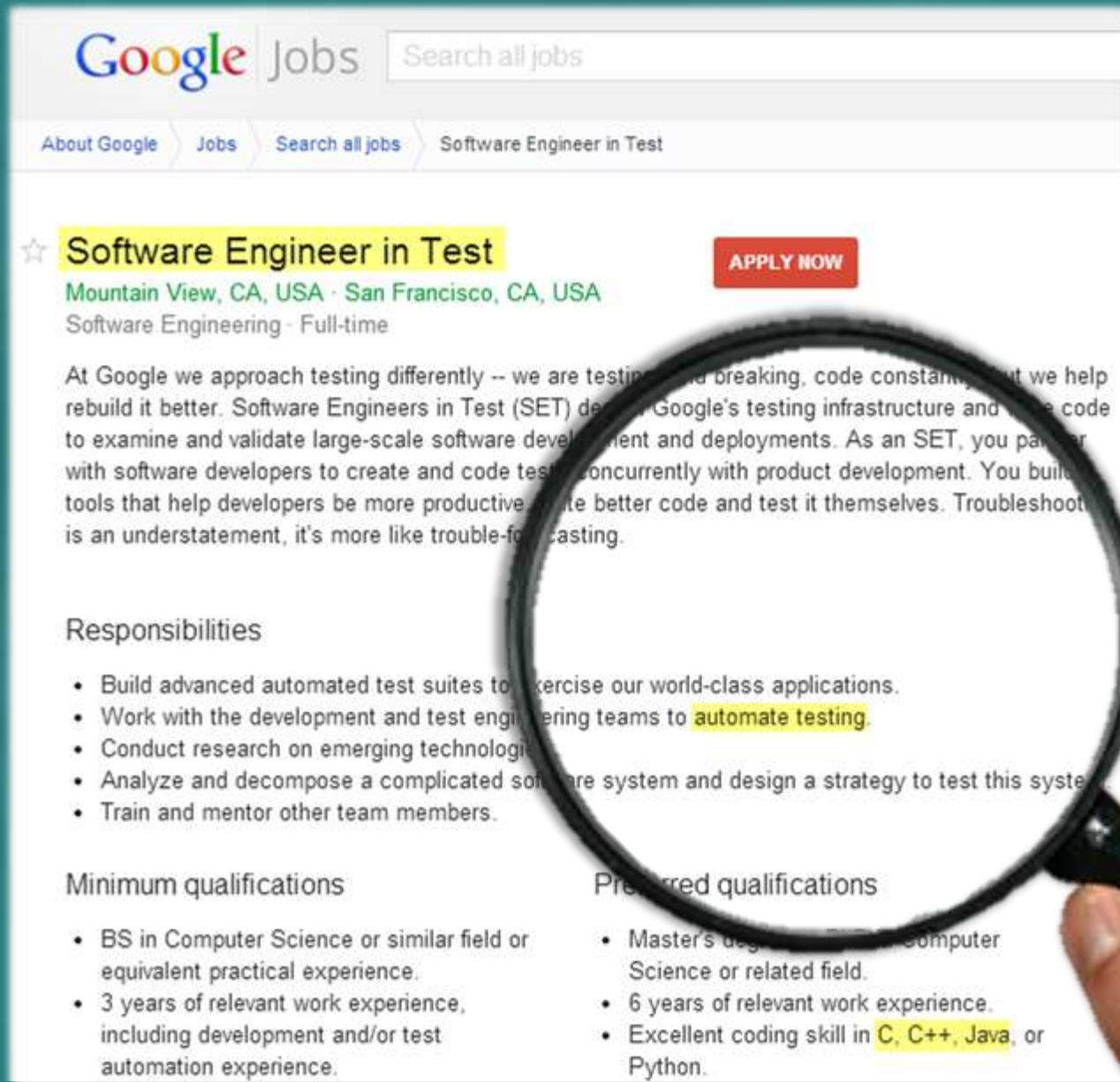
- Build advanced automated test suites to support world-class applications.
- Work with the development and test engineering teams to automate testing.
- Conduct research on emerging technologies to improve testing.
- Analyze and decompose a complicated software system.
- Train and mentor other team members.

Minimum qualifications

- BS in Computer Science or similar field or equivalent practical experience.
- 3 years of relevant work experience, including development and/or test automation experience.
- Master's degree in Computer Science or related field.
- 6 years of relevant work experience.
- Excellent coding skill in C, C++, Java, or Python.

So, don't rule out testing as a career path!

Testing as a career...?



The image shows a screenshot of a Google Jobs listing for a 'Software Engineer in Test' position. A magnifying glass is held over the 'Responsibilities' section, highlighting the text. The listing includes the job title, location (Mountain View, CA, USA - San Francisco, CA, USA), and a description of the role. The 'Responsibilities' section lists five bullet points, and the 'Minimum qualifications' section lists three bullet points. The 'Preferred qualifications' section lists three bullet points.

Google Jobs Search all jobs

About Google Jobs Search all jobs Software Engineer in Test

☆ **Software Engineer in Test** **APPLY NOW**

Mountain View, CA, USA - San Francisco, CA, USA
Software Engineering - Full-time

At Google we approach testing differently -- we are testing and breaking, code constantly, but we help rebuild it better. Software Engineers in Test (SET) design Google's testing infrastructure and write code to examine and validate large-scale software development and deployments. As an SET, you partner with software developers to create and code test concurrently with product development. You build tools that help developers be more productive, write better code and test it themselves. Troubleshooting is an understatement, it's more like trouble-forecasting.

Responsibilities

- Build advanced automated test suites to exercise our world-class applications.
- Work with the development and test engineering teams to **automate testing**.
- Conduct research on emerging technologies.
- Analyze and decompose a complicated software system and design a strategy to test this system.
- Train and mentor other team members.

Minimum qualifications

- BS in Computer Science or similar field or equivalent practical experience.
- 3 years of relevant work experience, including development and/or test automation experience.

Preferred qualifications

- Master's degree in Computer Science or related field.
- 6 years of relevant work experience.
- Excellent coding skill in **C, C++, Java**, or Python.



Your PC ran into a problem and needs to restart. We're just collecting some error info, and then we'll restart for you.

41% complete



For more information about this issue and possible fixes, visit <https://www.windows.com/stopcode>

If you call a support person, give them this info:

Stop code: CRITICAL_PROCESS_DIED

No need to celebrate!
This is a simulation 😊



Week 10

Topics:

- [W10.1] **Design Patterns** ⚡⚡
- [W10.2] **Defensive Programming** ⚡⚡
- [W10.3] **Test Cases: Intro** ⚡⚡
- [W10.4] **Test Cases: Equivalence Partitioning** ⚡⚡
- [W10.5] **Test Cases: Boundary Value Analysis** ⚡⚡

Admin:

1. Submit weekly quiz **P**
2. Take part in the CATcher load testing
⌚ during the briefing on Mar 28th **P**

tP: **v1.4**

1. 👤 Do a postmortem of the previous iteration
2. 👤 Plan the alpha version (**v1.4**)
3. 👤 Deliver the alpha version (**v1.4**)
4. 👤 Smoke-test CATcher **⌚ COMPULSORY** **P**
5. 👤 Start updating UML diagrams in the DG

Week 10

Topics:

- [W10.1] **Design Patterns** ⚡⚡
- [W10.2] **Defensive Programming** ⚡⚡
- [W10.3] **Test Cases: Intro** ⚡⚡
- [W10.4] **Test Cases: Equivalence Partitioning** ⚡⚡
- [W10.5] **Test Cases: Boundary Value Analysis** ⚡⚡

Admin:

1. Submit weekly quiz **P**
2. Take part in the CATcher load testing
⌚ during the briefing on Mar 28th **P**

tP: **v1.4**

1. 👤 Do a postmortem of the previous iteration
2. 👤 Plan the alpha version (**v1.4**)
3. 👤 Deliver the alpha version (**v1.4**)
4. 👤 Smoke-test CATcher **⌚ COMPULSORY** **P**
5. 👤 Start updating UML diagrams in the DG

Add all target features in v1.4 !